OS Lab: Programs & Viva Guide

Experiments 1-10 | two short disk scheduling programs | interview-style questions with answers

Based on the supplied ECE Operating System Lab Manual (2025). The C programs below retain the manual's arrays and algorithm, with missing headers added where needed.

Start here: the two programs

1. FCFS disk scheduling - shortest and easiest

FCFS serves requests in the given order. 53 is the starting position of the disk head; it does not have to appear in the request list. For each request: add the distance, then move the head.

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int req[] = {98, 183, 37, 122, 14, 124, 65, 67};
    int n = sizeof(req) / sizeof(req[0]);
    int head = 53, total = 0;

    for (int i = 0; i < n; i++) {
        total += abs(req[i] - head);
        head = req[i];
    }

    printf("FCFS total head movement = %d\n", total);
    return 0;
}
```

Output: FCFS total head movement = 640

What is n? `sizeof(req) / sizeof(req[0])` divides the size of the entire array by the size of one element. It counts 8 requests. Using n means the loop still works if you edit the array.

Why 53? The head begins at track 53. The first move is 53 to 98, so add |98 - 53| = 45. Then set head to 98 before processing 183. The full path is 53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67.

Memory cue: Add distance → move head → repeat.

Small manual fix: Add `#include <stdlib.h>`, which declares `abs()`.

2. SSTF disk scheduling - second short option

SSTF chooses the closest request that has not been served. It uses the same request list and starting head position as FCFS.

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

int main(void) {
    int req[] = {98, 183, 37, 122, 14, 124, 65, 67};
    int n = sizeof(req) / sizeof(req[0]);
    int visited[8] = {0};
    int head = 53, total = 0;

    for (int k = 0; k < n; k++) {
        int min = INT_MAX, idx = -1;
        for (int i = 0; i < n; i++) {
            if (!visited[i] && abs(req[i] - head) < min) {
                min = abs(req[i] - head);
                idx = i;
            }
        }
        visited[idx] = 1;
        total += min;
        head = req[idx];
    }

    printf("SSTF total head movement = %d\n", total);
    return 0;
}
```

Output for the printed requests: SSTF total head movement = 236. The manual prints 208, but the code and request list produce 236.

Trace: 53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183. Add the absolute distances between neighbours.

Memory cue: Find nearest → mark visited → add distance → move head.

Viva questions and short answers

Practise saying these answers aloud. The questions cover both the programs and the OS concepts behind them.

Experiment 1 - Installing an OS using VMware

1. **What is a virtual machine?**
   A software-created computer that runs an operating system using resources from a real computer.
2. **What are host and guest OS?**
   The host runs on the physical computer. The guest runs inside the VM; for example, Windows as host and Ubuntu as guest.
3. **What does a hypervisor do?**
   It creates and manages VMs and assigns virtual CPU, memory, storage and other hardware. VMware Workstation is a hypervisor.
4. **Why use an ISO file?**
   It is an image of installation media from which the VM can boot to install the guest OS.
5. **Does allocating 4 GB RAM to a VM create extra RAM?**
   No. It comes from the host machine, which must have enough for both systems.

6. **What is a snapshot?**
   A saved VM state that can be restored later. It is useful before risky changes, but it is not a substitute for a separate backup.
7. **Why install VMware Tools or open-vm-tools?**
   They improve integration such as display resizing, mouse behaviour and shared folders.

Experiment 2 - Linux commands and shell scripts

1. **What is a shell?**
   A program that accepts commands and executes them. Bash is one example; a shell script contains commands to run.
2. **How do pwd and ls differ?**
   pwd prints the path of the current directory; ls lists directory contents.
3. **How do cp and mv differ?**
   cp copies a file; mv moves or renames the original.
4. **What does chmod 755 hello.sh mean?**
   The owner has read, write and execute permission (7); group and others have read and execute (5 each).
5. **What are $1 and $2 in the backup script?**
   The first and second command-line arguments: source and destination in that script.
6. **Why write "$src" instead of $src?**
   Quotes keep a value containing spaces together and prevent unwanted word splitting.
7. **What does -f "$fname" check?**
   Whether that path exists and is a regular file. It does not search every directory.
8. **What is a PID, and what does kill -9 do?**
   A PID identifies a process. kill -9 sends SIGKILL to terminate it forcibly; the process cannot handle that signal to clean up.

Experiment 3 - CPU scheduling

Formulas: Turnaround time (TAT) = Completion time (CT) - Arrival time (AT). Waiting time (WT) = TAT - Burst time (BT).

1. **What is CPU scheduling?**
   Choosing which ready process gets the CPU next.
2. **Preemptive versus non-preemptive?**
   Preemptive scheduling can interrupt a running process; non-preemptive scheduling lets it continue until the burst ends or it blocks.
3. **How does FCFS work?**
   Run processes in arrival order. A long process can delay short ones; this is the convoy effect.
4. **How do SJF and SRTF differ?**
   SJF chooses the shortest available burst without preemption. SRTF chooses the shortest remaining time and can preempt.
5. **Can SJF or SRTF cause starvation?**
   Yes. Repeatedly arriving short jobs may keep a long job waiting.

6. **How does Round Robin work?**
   Each ready process runs for at most one time quantum, then goes back to the queue if unfinished.
7. **What if the time quantum is too small or too large?**
   Too small: frequent context switches and overhead. Very large: Round Robin approaches FCFS.
8. **Waiting time versus response time?**
   Waiting time is total time in the ready queue. Response time is arrival to first CPU service.

Experiment 4 - Producer-consumer using semaphores

1. **What is the producer-consumer problem?**
   Producers insert into a shared bounded buffer; consumers remove from it. They must avoid full-buffer writes and empty-buffer reads.
2. **What is a semaphore?**
   A synchronization variable controlled by wait and post operations. wait may block; post signals availability.
3. **What are empty and full initially?**
   empty counts free slots and starts at 5; full counts filled slots and starts at 0.
4. **Why is a mutex also needed?**
   The semaphores count available slots. The mutex protects shared buffer contents and indexes during changes.
5. **What is a race condition?**
   A result that depends on unpredictable timing of concurrent accesses to shared data.
6. **Why use modulo BUF_SIZE for in and out?**
   It wraps an index to zero after the last slot, making a circular buffer.
7. **Why does the consumer wait on full?**
   It must not read until at least one item has been produced.
8. **Is the print order identical on every run?**
   No. Thread scheduling can change the order, even when synchronization is correct.

Experiment 5 - Banker’s algorithm

1. **What does Banker’s algorithm do?**
   It avoids deadlock by checking whether a proposed resource allocation leaves the system in a safe state.
2. **Define Allocation, Max, Need and Available.**
   Allocation: resources held; Max: declared maximum; Need = Max - Allocation; Available: currently free resources.
3. **What is a safe state?**
   There is some order in which every process can finish with available resources and resources released by preceding processes.
4. **What is a safe sequence?**
   That order of processes. When Need <= Available, a process can finish and release its Allocation.
5. **Does unsafe mean already deadlocked?**
   No. Unsafe means deadlock could happen; deadlock means processes are currently unable to proceed.
6. **Avoidance versus detection?**
   Avoidance checks before granting resources. Detection checks for a deadlock after allocation.

7. **What is a practical limitation?**
   Processes must state their maximum resource needs in advance, which is often difficult.

Experiment 6 - Deadlock detection

1. **What is deadlock?**
   Processes remain blocked while waiting for resources that cannot become available as they are currently held.
2. **What four conditions make deadlock possible?**
   Mutual exclusion, hold and wait, no preemption, and circular wait.
3. **How does this differ from Banker’s algorithm?**
   Banker’s uses maximum future needs to avoid unsafe states; detection examines current outstanding requests.
4. **What if a process’s request can be met?**
   Treat it as able to finish, return its Allocation to Available, and try the others again.
5. **Why repeat the scan?**
   One process finishing may release resources needed by another.
6. **What if no more progress is possible?**
   The processes still unable to finish are reported as deadlocked for the given state.
7. **How can the OS recover?**
   Terminate processes or preempt resources where possible; either approach can lose work or hurt performance.

Experiment 7 - Page replacement and paging

1. **What is paging?**
   Virtual memory is divided into fixed-size pages, and physical memory into fixed-size frames.
2. **What is a page fault?**
   A referenced page is absent from physical memory and must be brought in.
3. **What does FIFO replace?**
   The page that entered memory earliest, regardless of recent use.
4. **What does LRU replace?**
   The page that has not been used for the longest time.
5. **What does Optimal replace?**
   The page whose next use is farthest in the future, or that will never be used again. It is a benchmark.
6. **What is Belady’s anomaly?**
   Under FIFO, adding frames can sometimes increase page faults. LRU and Optimal do not show it.
7. **Page hit versus page fault?**
   A hit finds the page in memory; a fault requires bringing it into memory.
8. **What is thrashing?**
   Spending excessive time moving pages between memory and storage because active pages do not fit in available frames.

Experiment 8 - Memory allocation methods

1. **What is the goal of these methods?**
   Choose a suitable free memory block for a process.
2. **What is First Fit?**
   Pick the first block large enough when scanning from the beginning.
3. **What is Next Fit?**
   Resume searching from the point where the previous search stopped.
4. **What is Best Fit?**
   Pick the smallest block that can still hold the process.
5. **What is Worst Fit?**
   Pick the largest available block.
6. **Is Best Fit always best overall?**
   No. It can leave many small, unusable gaps.
7. **What is external fragmentation?**
   Free memory exists in separate pieces, but no single piece is large enough for a request.
8. **What does Not Allocated mean?**
   No currently available individual block fits the process under that method.

Experiment 9 - Disk scheduling

1. **Why schedule disk requests?**
   On a mechanical hard disk, request order affects head travel distance and waiting time.
2. **What is FCFS disk scheduling?**
   Serve requests in their given order. It is simple but can involve considerable head movement.
3. **What is SSTF?**
   Choose the pending request closest to the current head position.
4. **What is SSTF's drawback?**
   A faraway request may starve if nearby requests keep arriving.
5. **What is SCAN?**
   The head moves in one direction serving requests, then reverses; it is the elevator algorithm.
6. **SCAN versus LOOK?**
   Usually SCAN reaches the disk end before reversing; LOOK reverses at the final request in that direction.
7. **What does C mean in C-SCAN and C-LOOK?**
   Circular: service in one direction, return to the other side, then continue in the original servicing direction.
8. **Do head movement calculations apply equally to SSDs?**
   No. SSDs have no mechanical read/write head or seek movement.

Printed manual discrepancy: Its SSTF sample says 208; executing the supplied code with its supplied requests and head = 53 gives 236.

Experiment 10 - Threads and IPC

1. **Process versus thread?**
   A process has its own address space; threads of the same process share memory but have separate execution state and stacks.
2. **What do pthread_create and pthread_join do?**
   create starts a thread; join waits for a specified thread to finish.
3. **Why pass the worker an ID address?**
   The worker accepts void *, casts it back to int *, and reads the ID.
4. **What is IPC?**
   Inter-process communication: mechanisms that let separate processes exchange information.
5. **Pipe versus named pipe (FIFO)?**
   An ordinary pipe commonly connects related processes. A named pipe has a filesystem name and can connect unrelated processes.
6. **What does fork return?**
   0 in the child, the child PID in the parent, and -1 on failure.
7. **Why close unused pipe ends?**
   The child only reads and parent only writes. Closing unused ends also permits correct end-of-stream behaviour.
8. **Why is shared memory fast, and what must be managed?**
   Processes access the same mapped memory without copying each message through a pipe, but must synchronize access to avoid races.
9. **What is a socket pair?**
   Two connected local sockets, capable of communication in both directions.

Final five to practise aloud

Process vs thread: Separate address spaces vs shared address space within one process.

Preemptive vs non-preemptive: The OS may interrupt the running CPU burst vs it waits for the burst to end or block.

Deadlock avoidance vs detection: Check before allocation vs identify deadlock afterward.

Page fault vs page hit: Page absent from memory vs page already there.

Why synchronize shared data?: To prevent race conditions when execution order changes.